TCS-504: System Design — Assignment 1

Movie Ticket Booking System (Single Cinema Architecture)

Author: Dhruv Bhatt | Branch: B.Tech. CSE (5th Sem) **Date: 07-Sept-2026 | Code: TCS-504**

1. Executive Summary & Problem Scope

This project implements a complete Object-Oriented, menu-driven movie ticket booking engine for a single cinema (e.g., PVR or INOX) in C++14. The system is designed with strict adherence to SOLID design principles, GoF Strategy Pattern, and Clean Code guidelines under the academic course rule: *'One class per file, No header files'*.

8 Mandatory Core Features Built (F1 – F8):

- **F1:** List all movies currently playing with runtime & language.
- **F2:** For a chosen movie, list its screening shows (auditorium screen + start time).
- **F3:** For a chosen show, display seat layout with AVAILABLE [ ] / BOOKED [X] status.
- **F4:** Book one or more seats for a show (atomic validation rejects whole order if any seat is booked).
- **F5:** Dynamic pricing by tier: **SILVER Rs.150**, **GOLD Rs.250**, **PLATINUM Rs.400**.
- **F6:** Pay by UPI, Card, or Cash (failed payment aborts booking & triggers atomic seat rollback).
- **F7:** Print formatted ticket: Booking ID, Movie, Screen, Start Time, Seats, Total Amount, Status.
- **F8:** Cancel a booking — all associated seats immediately revert to AVAILABLE.
- **Bonus (F9):** Polymorphic Seat Tier Inspector for real-time tier metrics (Total, Booked, Free).

2. Step A — Requirement Analysis (FR + NFR)

Functional Requirements (FR):

- **FR1 — Movie Listing:** The system shall list all movies playing with title, language, and duration in minutes.
- **FR2 — Show Scheduling:** For any chosen movie, the system shall list scheduled shows with screen name and start time.
- **FR3 — Seat Layout Display:** For any chosen show, the system shall display the physical seat layout categorized by tiers (SILVER, GOLD, PLATINUM) with real-time status [ ] (Available) or [X] (Booked).
- **FR4 — Booking (Standard):** A customer selects one or more seat numbers for a show. If any selected seat is already BOOKED or invalid, the whole booking is rejected and no seat changes state. Booking is confirmed only after payment succeeds.
- **FR5 — Seat Tier Pricing:** The system shall calculate the total booking cost dynamically based on seat types: SILVER = Rs.150, GOLD = Rs.250, PLATINUM = Rs.400.
- **FR6 — Payment (Standard):** Exactly one method (UPI / Card / Cash) per booking. If payment fails, seats are released and booking status becomes FAILED.
- **FR7 — Ticket Generation:** Upon successful payment, the system shall format and print an itemized ticket showing Booking ID, Movie, Screen, Start Time, Booked Seats, Total Amount, and Status (CONFIRMED).
- **FR8 — Booking Cancellation:** A customer can cancel a booking using its Booking ID, updating status to CANCELLED and restoring seats to AVAILABLE.

Non-Functional Requirements (NFR):

- **NFR1 — Modularity:** Strict 'one class per file' modularity without external header files.
- **NFR2 — Extensibility (OCP):** Adding new payment options (e.g. NetBanking) requires adding a class without modifying existing orchestrator code.
- **NFR3 — Robust Input Validation:** Gracefully handles invalid seat IDs (e.g. Z9) and numerical menu strings without crashing.
- **NFR4 — Transactional Rollback:** Operations maintain atomic consistency; failed payment rolls back temporary seat locks.

3. Step B — Noun–Verb Analysis

Noun Found	Keep as Class?	Design Rationale & Justification
Movie	**Yes**	Core entity holding domain data (title, language, duration) and identity.
Seat	**Yes**	Represents physical chair in auditorium with fixed number and seat tier.
ShowSeat	**Yes**	**Crucial distinction:** Dynamic availability status of a seat for a specific showtime.
Screen	**Yes**	Auditorium entity that contains and owns physical Seat objects.
Cinema	**Yes**	Top-level theatre entity that owns auditorium screens.
Show	**Yes**	Screening entity binding Movie, Screen, and StartTime; owns ShowSeats.
Customer	**Yes**	Core actor entity holding customer identity (name, phone number).
Booking	**Yes**	Transactional record encapsulating booking ID, seats, total, and status.
Payment	**Yes**	Abstract strategy defining the payment contract for runtime polymorphism.
PriceCalculator	**Yes**	Pure domain calculation service dedicated solely to pricing logic.
TicketPrinter	**Yes**	Pure presentation service responsible solely for console formatting.
BookingService	**Yes**	Domain orchestrator managing end-to-end booking workflows and rollback.
SeatCategory	**Yes**	Abstract base class for polymorphic tier inspection and statistics.
seat layout	**No**	Visual representation view → encapsulated as method in TicketPrinter.
menu / console	**No**	Execution entry point & UI loop → placed in main.cpp.

4. Step C — Class Responsibilities & Specifications

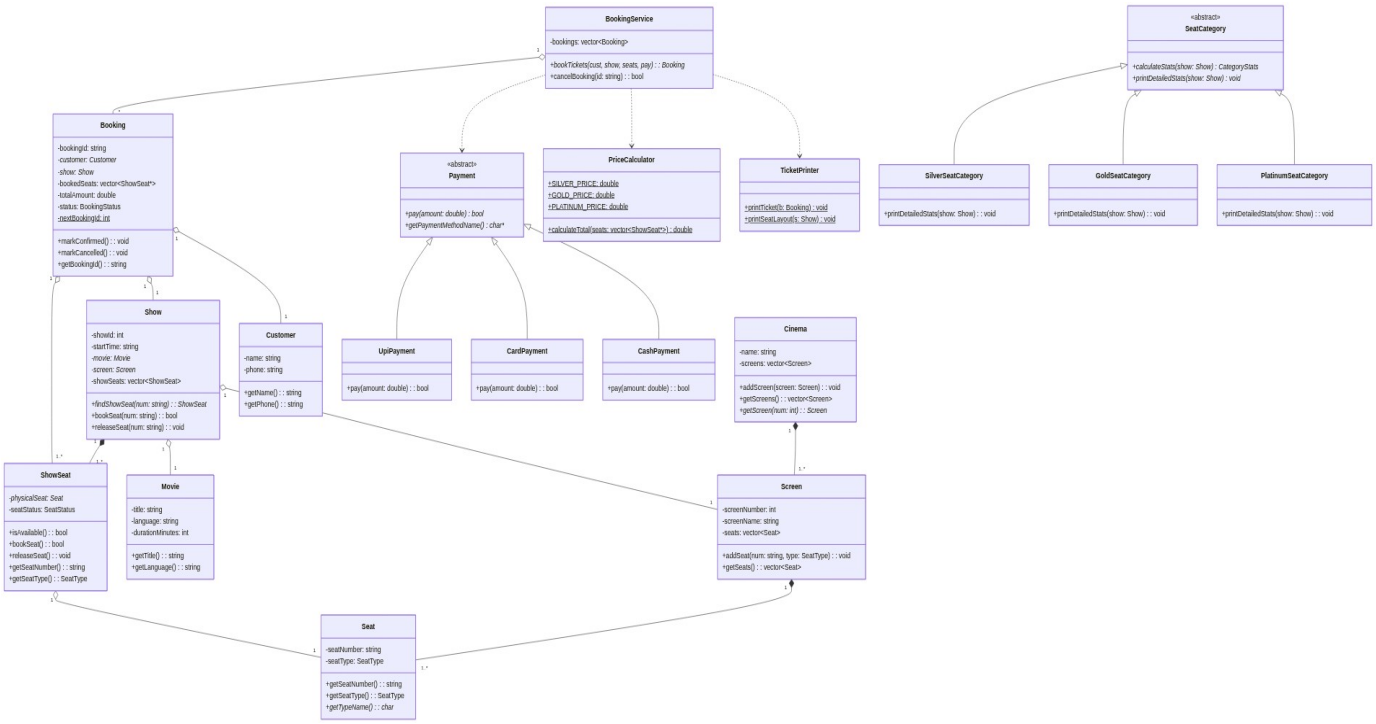
Architectural Decision: Why ShowSeat and NOT Just Seat?
Physical chair A1 exists exactly once in an auditorium (Screen-1). However, its availability status changes for every show: it may be **BOOKED** for 06:00 PM and **AVAILABLE** for 09:00 PM. If status were stored in Seat, booking A1 for one show would incorrectly block it for all shows. Therefore, Seat represents the physical chair, while ShowSeat represents the temporal availability status for a specific Show.

Class	Data Members (State)	Member Methods (Behavior)	What it MUST NOT Do
Movie	title, language, durationMinutes	getTitle(), getLanguage(), getDurationMinutes()	Must NOT know about screens or pricing.
Seat	seatNumber, seatType	getSeatNumber(), getSeatType(), getTypeName()	Must NOT track booking availability.
Screen	screenNumber, screenName, seats	addSeat(), getSeats(), getScreenNumber()	Must NOT manage show scheduling.
Cinema	name, screens	addScreen(), getScreens(), getScreen()	Must NOT process ticket bookings.
Show	showId, startTime, movie, screen, showSeats	findShowSeat(), bookSeat(), releaseSeat()	Must NOT compute monetary totals.
ShowSeat	physicalSeat, seatStatus	isAvailable(), bookSeat(), releaseSeat()	Must NOT create physical chairs.
Customer	name, phone	getName(), getPhone()	Must NOT directly modify seat statuses.
Booking	bookingId, customer, show, bookedSeats, totalAmount, status	markConfirmed(), markFailed(), markCancelled()	Must NOT print console output.
Payment	*(Pure Virtual Interface)*	pay(amount) = 0, getPaymentMethodName()	Must NOT store booking histories.
PriceCalculator	SILVER_PRICE, GOLD_PRICE, PLATINUM_PRICE	getPriceForSeatType(), calculateTotal()	Must NOT mutate seat availability.
TicketPrinter	*(Pure Presentation Utility)*	printTicket(), printSeatLayout()	Must NOT mutate business state.
BookingService	bookings: vector<Booking>	bookTickets(), cancelBooking(), findBooking()	Must NOT format UI art.

5. Step D — Relationships & Lifetime Test

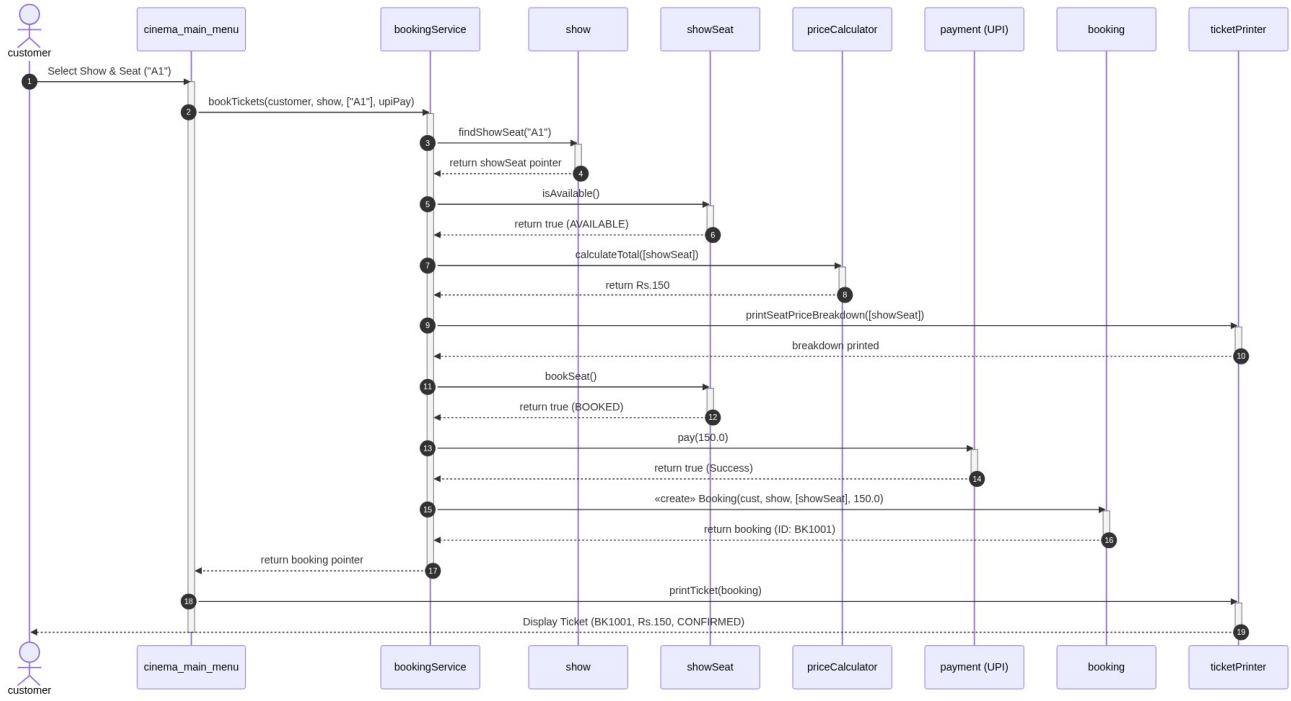
Pair	Relationship	Lifetime Test & Architectural Justification
Cinema — Screen	**Composition (◆)**	**YES, the part dies.** If Cinema is demolished, all Screen auditoriums inside it cease to exist.
Screen — Seat	**Composition (◆)**	**YES, the part dies.** Physical seats are bolted to the auditorium floor; destroyed with the Screen.
Show — Movie	**Aggregation (■)**	**NO, the part lives.** If a show is cancelled, the Movie entity still exists in the cinema catalogue.
Show — Screen	**Aggregation (■)**	**NO, the part lives.** When a show finishes, the Screen auditorium remains intact for other shows.
Show — ShowSeat	**Composition (◆)**	**YES, the part dies.** ShowSeat is tied to that specific screening slot; dies with the Show.
Booking — Customer	**Aggregation (■)**	**NO, the part lives.** Customer profile exists independently of whether a specific booking exists.
Booking — ShowSeat	**Aggregation (■)**	**NO, the part lives.** If a Booking is cancelled, ShowSeats remain in the Show (reset to AVAILABLE).
Booking — Payment	**Association (→)**	**Independent lifeline.** Booking delegates payment authorization to Payment during checkout.
Payment — UpiPayment	**Inheritance (⇒)**	**Is-A Relationship.** UpiPayment implements the abstract Payment interface.
BookingService — Booking	**Aggregation (■)**	**Maintains collection.** BookingService orchestrates and stores confirmed Booking entities.

6. Step E — UML Class Diagram



7. Step F — UML Sequence Diagram

Use Case: Customer books 1 seat and pays by UPI (Lifelines: Customer, UI, BookingService, Show, ShowSeat, PriceCalculator, Payment, Booking, TicketPrinter)



8. Step G — SOLID Principles & Intentional Design Non-Goals

Principle	How Applied in Codebase	Concrete Code Location
S — Single Responsibility	Each class has one reason to change: PriceCalculator prices; TicketPrinter prints; BookingService orchestrates.	11_PriceCalculator.cpp 12_TicketPrinter.cpp
O — Open/Closed	Adding a new payment method (e.g. NetBanking) requires adding a subclass without editing existing code.	09_Payment.cpp 10_PaymentTypes.cpp
L — Liskov Substitution	Any derived payment class (UpiPayment, CardPayment) can be passed as Payment* with consistent behavior.	10_PaymentTypes.cpp
I — Interface Segregation	Payment interface defines only essential payment contract, avoiding non-universal methods.	09_Payment.cpp
D — Dependency Inversion	BookingService depends on abstract Payment* interface rather than concrete payment types.	13_BookingService.cpp

Intentional Design Non-Goals (What Was Deliberately NOT Done):

- **Deliberately Did NOT Create a Separate `SeatLayout` Class:** A seat layout is a visual presentation concern rather than an independent domain entity with its own lifecycle. Encapsulating it in TicketPrinter keeps domain models clean and adheres to High Cohesion.
- **Deliberately Did NOT Let `Booking` Own `Payment`:** Decoupling payment processors from persistent booking records preserves security and transactional independence.

9. Step H — OOP Concepts Mapping & Clean Code Checklist

OOP Concept	Implementation Details	Code Reference
1. Encapsulation	Private attributes; mutation restricted to validated methods (bookSeat, releaseSeat).	06_ShowSeat.cpp:24-60 08_Booking.cpp:29-76
2. Abstraction	Abstract base class with pure virtual method virtual bool pay(double amount) = 0.	09_Payment.cpp:14-22
3. Inheritance	UpiPayment, CardPayment, CashPayment inherit from abstract Payment base.	10_PaymentTypes.cpp:19
4. Runtime Polymorphism	Dynamic method dispatch: payment->pay(totalAmount).	13_BookingService.cpp:91
5. Compile-Time Polymorphism	Overloaded constructors and overloaded PriceCalculator::calculateTotal() methods.	11_PriceCalculator.cpp:33-46
6. Static Members	Class-level static counter static int nextBookingId generating unique IDs (BK1001).	08_Booking.cpp:30-46
7. 'this' Keyword	Used to disambiguate member variables from parameters and refer to calling object.	01_Movie.cpp, 07_Customer.cpp
8. Composition	Cinema owns Screen; Screen owns Seat; Show owns ShowSeat.	03_Screen.cpp, 04_Cinema.cpp
9. Aggregation	Show references Movie & Screen; Booking references Customer & ShowSeats.	05_Show.cpp, 08_Booking.cpp
10. Association	Customer interacts with BookingService to book tickets; neither owns the other.	13_BookingService.cpp:69-74

10. Step I — Automated Verification Test Suite Execution

```
======================================================
RUNNING OPTIMIZED AUTOMATED VERIFICATION SUITE
======================================================

Screen-1 06:00 PM | 3 Idiots
SILVER A1[ ] A2[X] A3[ ] A4[ ]
GOLD B1[ ] B2[ ] B3[X]
PLATINUM C1[ ] C2[ ]

Selected Seats Breakdown:
A1 SILVER Rs.150
B2 GOLD Rs.250
TOTAL Rs.400

[UPI] Rs.400 paid successfully

================ TICKET ================
Booking ID : BK1001
Movie : 3 Idiots
Screen : Screen-1 06:00 PM
Seats : A1, B2
Amount : Rs.400 Status: CONFIRMED
========================================

>> TEST 1 PASSED: Confirmed ID BK1001
[Booking Rejected] Seat A1 is already BOOKED. No seats reserved.
>> TEST 2 PASSED: Double booking rejected; A3 remains AVAILABLE.

Selected Seats Breakdown:
A3 SILVER Rs.150
TOTAL Rs.150

[Card] Transaction FAILED: Insufficient balance / Card declined.
[Booking Failed] Payment was not successful. Seats have been released.
>> TEST 3 PASSED: Payment failure rolled back; A3 remains AVAILABLE.
[Success] Booking BK1001 has been cancelled. Seats are now AVAILABLE.
>> TEST 4 PASSED: Booking cancelled; seats A1 & B2 released.
[Booking Rejected] Seat Z9 does not exist.
>> TEST 5 PASSED: Invalid seat handled without crash.

======================================================
ALL OPTIMIZED TESTS PASSED SUCCESSFULLY
======================================================
```